Movie Ticketing System

Software Requirements Specification

Version 1.0

February 15, 2026

Group 1
Elisa Han, Ruhi Singh, Ella Lamie

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
2-15-26	Version 1	Elisa Han, Ruhi Singh, Ella Lamie	First Revision
2-18-26	Version 2	Elisa Han, Ruhi Singh, Ella Lamie	Edits

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Analysis Models 4

4.1 Sequence Diagrams 5
4.3 Data Flow Diagrams (DFD) 5
4.2 State-Transition Diagrams (STD) 5

5. Change Management Process 5

A. Appendices 5

A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

This SRS Document includes information about the Movie Ticketing System. Section 1 includes the purpose, scope, any definitions/acronyms/abbreviations the user needs to know, references, and a general overview. Section 2 contains the general description, the product perspective in relation to other products, the functions the product will perform, the user characteristics, constraints, and assumptions/dependencies. Section 3 provides an overview of requirements such as the external interface requirements, functional requirements, use cases, classes and objects, non-functional requirements, inverse requirements, design constraints, logical database requirements, and any other requirements. Section 4 goes over analysis models, including sequence diagrams, data flow diagrams, and state transition diagrams. Section 5 goes over the change management process. The appendices include any additional information. The system is needed in order to make ticket buying easier for both employees and customers, which should be the client's goal as work hours and customer complications will be to a minimum. The major features of the Movie Ticketing System are movie browsing, showtime listings, and purchasing and payment support.

1.1 Purpose

The purpose of this SRS is to outline the requirements, functionality, and constraints of building a theatre ticketing system, enabling a software engineer to design and develop an adequate and efficient web-based software system. Administrators can use this SRS as a guide for how the Movie Ticketing System works.

1.2 Scope

The Movie Ticketing System allows users, scaling at least 10,000 people simultaneously, to purchase tickets on a data-driven web browser, maintaining a response time of less than 2 seconds per user request. This browser will display the database of movie showtimes, tickets available, and open seats. Seat availability will update in real time to prevent double-booking. It will be able to generate a ticket ID for every ticket purchase in order to prevent scalping. The Movie Ticketing System will be up for 99.99% of the year, excluding scheduled maintenance periods every two months. It will prevent 98% of ticket scalping. The system will be used by customers and administrators in order to purchase and keep track of movie tickets, respectively.

1.3 Definitions, Acronyms, and Abbreviations

- Administrator: An employee that manages movies, showtimes, seating, and customer ticket information.
- Customer: A user that browses movies, selects show times and seating, and makes payments through the system.
- Movie showtimes: A scheduled date and time in which the movie will be taking place. Relating to specific movie and movie theaters.
- Ticket ID: a unique identification generated by the system to represent a purchased ticket.

- HTTPS (HyperText Transfer Protocol Secure): A secure communication protocol used to encrypt data transmitted between the user and the system.

1.4 References

Bandakkanavar, R. (2023, May 8). *Software Requirements Specification document with example - Krazytech*. Krazytech. https://krazytech.com/projects/sample-software-requirements-specificationsrs-report-airline-database

Institute Of Electrical And Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications.* Ieee.

These sources were not used as a reference throughout the SRS, but rather used as a guide on how to properly format the SRS.

1.5 Overview

Section 1 provides an introduction to the SRS, including the purpose, the scope, definitions, and structure of the document.

Section 2 gives a general description of the system, including user characteristics, assumptions, constraints, and overall system environment.

Section 3 provides the specific requirements of the system. This is the primary reference for interface design, including user interface, hardware and software. There are detailed functional requirements that map out the detailed features of the system and how the user is to interact with them.

Section 4 presents analysis models such as sequence diagrams, data flow diagrams, and state-transition diagrams that support the requirements defined in Section 3.

Section 5 describes the change management process used to update and maintain this SRS throughout its life cycle

This SRS is organized to move from general system requirements and concepts to detailed and testable features and requirements for clarity and traceability.

2. General Description

The Movie Ticketing System is affected by the implementation software, the hardware limitations, and the website it is connected to. An overview of its requirements is provided below, but this section does not state specific requirements.

Requirements:
- Website for implementation
- 24/7 availability
- Safety and security
- Database connection

2.1 Product Perspective

The Movie Ticketing System stores the following information:
- Information about ticket buyer
  It includes name, age, and ID for buying tickets.
- Information about movies
  It includes names of movies out, dates released, availability, and theater numbers. This is so the customer can choose what to watch.
- Ticketing ID system
  This is a custom ID for every ticket in order to prevent scalping.

The Movie Ticketing System is not independent and will be contained on each movie theater's website. Refer to Section 3.1 for interface requirements.

2.2 Product Functions

The Movie Ticketing System will have various functions:
Note that there is some overlap between the functions of the administrator and the functions of the customer.
From the administrator's side:
- getID will return a person's ID number
- getName will return a person's name
- getAge will return a person's age
  - All of these must be valid in order to proceed with reserveSeat
- addShowtime will let the administrator add a showtime for a movie
- removeShowtime will let the administrator delete a showtime for a movie
- editShowtime will let the administrator edit a showtime for a movie
- checkAvailability will let the administrator check how many seats are available
- editSeatAvailability will let the administrator edit the seat availability for a certain showtime

From the customer's side:
- checkAvailability will let the customer check how many seats are available
- reserveSeat will reserve a seat after having paid
- generateTicket will allow the user to generate their ticket after payment has been made
- deleteReservation will allow a user to cancel their seat reservation and ticket

2.3 User Characteristics

Users should be able to get information about movie availability and scheduling. The intended users are both customers and employees.
The customer should be able to:

- View all movies
- Choose a movie and a time
- Choose seating arrangements
- Schedule a movie ticket
- Cancel a movie ticket

The educational level and experience is not needed with customers.
However, the employee has all of the same functions as a user but including management functions:

- See all information about people who have scheduled a ticket
- Be able to edit movies, showtimes, and seat availability

The employee must have:

- At least a high school degree
- Must have experience with databases in order to make sense of the Movie Ticketing System

2.4 General Constraints

1. Hardware limitations that involve too many people trying to access the system at once
2. Reliability requirements in that if two people try to buy the last ticket at the exact same time, both might get it
3. Safety and security considerations in that someone might hack the system

2.5 Assumptions and Dependencies

- Assuming that the Movie Ticketing System will be used in coordination with a movie theater's website, the Movie Ticketing System will be dependent and change with the type of software the website is implemented on.

3. Specific Requirements

This section goes over the specific requirements needed to understand and implement the Movie Ticketing System, including the external interface requirements, the inverse requirements, the logical database requirements, the functional requirements, and the non-functional requirements. It lists each requirement with a priority level and includes various functions. It also includes three use cases and a list of the Movie Ticketing System's classes and objects, as well as design constraints.

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.1.1 High Priority- The Movie Ticketing System can be accessed via:

- A mobile device
- A computer (desktop/laptop)
- A tablet

3.1.1.2 High Priority- The Movie Ticketing System will show:

- List of available movies
- Showtimes for each available movie
- Seats available per showtimes
- "Movie is full" when user tries to get seats to a full movie

3.1.1.3 High Priority- What the user can select using their touch input, keyboard, or mouse:

- A showtime for a specific movie
- One or more available seats

` **3.1.1.4** Medium Priority- Confirmation screen before payment:

- Title of movie
- Time of movie
- Seats selected
- Price

3.1.2 Hardware Interfaces

3.1.2.1 High Priority- The Movie Ticketing System will operate on

- A mobile device
- A computer (desktop/laptop)
- A tablet

3.1.3 Software Interfaces

3.1.3.1 High Priority- The Movie Ticketing System will interact with a database to store

- Movies playing
- Showtimes of each movie
- Seats available and taken
- Tickets bought

3.1.3.2 High Priority- The Movie Ticketing System will partner with a payment processor in order to facilitate payments from customers

3.1.4 Communications Interfaces

3.1.4.1 High Priority- The Movie Ticketing System will use https

- Allows for more safety from the both the user's and the system's side

3.2 Functional Requirements

3.2.1 Movie Browsing

3.2.1.1 Introduction

- Allows the user to look at all available movies
- Allows the user to look at each of their available showtimes.

3.2.1.2 Inputs

- The database will keep track of the available movies and showtimes and will input that into the system.

- No inputs from the user.

3.2.1.3 Processing

- The Movie Ticketing System will be able to get movies, showtimes, and seat availability from the database.

3.2.1.4 Outputs

- A list of all available movies
- A list of all the corresponding showtimes.

3.2.1.5 Error Handling

- If the input from the database fails, display an error message: “Cannot load movies from database”

3.2.2 Seat Selection and Payment

3.2.2.1 Introduction

- Allows the user to select seats
- Allows the user to pay for the seats in advance.

3.2.2.2 Inputs

- Selected movie
- Showtime
- Payment information
- Seat(s).

3.2.2.3 Processing

- The Movie Ticketing System will make sure the seat is available, and reserve the seats for 30 minutes, which, if the user has not paid, will un-reserve the seats.
- If not, it will make sure the payment information is valid, send a request to the payment processor, and confirm the payment and seats.

3.2.2.4 Outputs

- A map of where the seats are in the theater
- A confirmation message saying that the user has paid
- A ticket ID.

3.2.2.5 Error Handling

- If the seat becomes unavailable, display an error message saying “The seat is no longer available. Please pick another seat”.
- If the payment was unsuccessful, display an error message saying “The payment was unsuccessful. Try again”.

3.2.3 Administration Access

3.2.3.1 Introduction

- Allows the administrator to access
    - Movies
    - Showtimes
    - Tickets
    - Seats
    - Personal information

3.2.3.2 Inputs

- Movie details
- Showtime details
- Seat available

3.2.3.3 Processing

- The administrator will be able to access all of the information of ticketholders
- May edit movies, showtimes, and seating if they see fit

3.2.3.4 Outputs

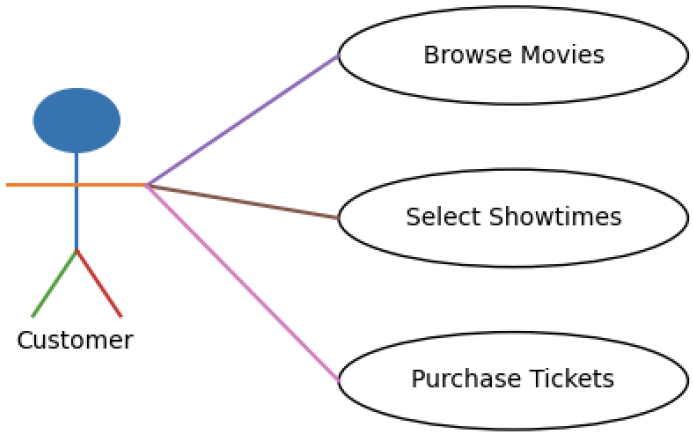
- A confirmation message will be shown after an edit is made to assure the administrator it went through

3.2.3.5 Error Handling

- If the edits are invalid, display an error message saying “Invalid inputs”

3.3 Use Cases

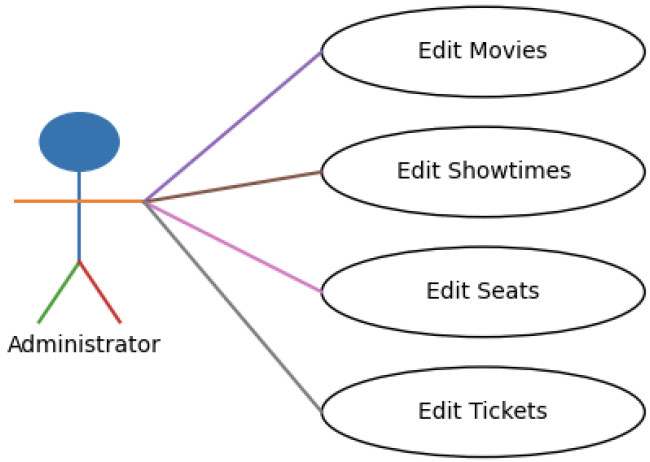
3.3.1 Use Case #1: The Customer’s Privileges



Description:

1. Customer can browse through the movies available
2. Customer can select the showtime they want for a particular movie
3. Customer can purchase their tickets

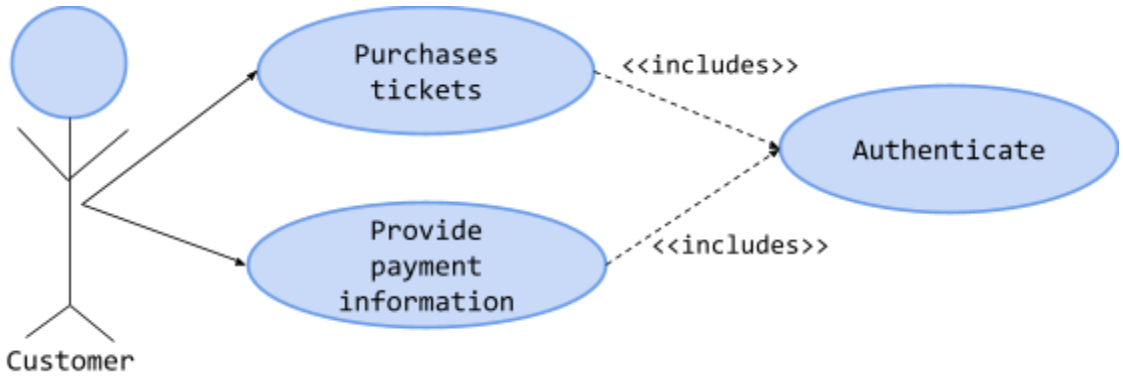
3.3.2 Use Case #2: The Administrator Privileges



Description:

1. Administrator can edit movies (i.e. adding or deleting movies)
2. Administrator can edit showtimes for a particular movie
3. Administrator can edit seats for a particular movie
4. Administrator can edit customer tickets

3.3.3 Use Case #3: Purchasing Tickets



Description:

1. Customers purchase tickets.
2. Customers provide payment after ticket claiming.
3. The system authenticates the order.

3.4 Classes / Objects

3.4.1 Movie

3.4.1.1 Attributes

- movieID
- movieName
- movieTime
- movieRating
- movieDescription

3.4.1.2 Functions

- getMovieID()
- getMovieName()
- getMovieTime()
- getMovieDescription()

```
Movie
- movieID : int
- movieName : String
- movieTime : String
- movieRating : double
- movieDescription : String

+ getMovieID()
+ getMovieName()
+ getMovieTime()
+ getMovieDescription()
```

References: 3.2.1 and 3.2.3

3.4.2 Showtimes

3.4.1.1 Attributes

- showtimeID
- movieID
- startTime

3.4.1.2 Functions

- getShowtimeID()
- getMovieID()
- checkAvailibilty()

```
Showtimes
- showtimeID : int
- movieID : int
- startTime : String

+ getShowtimeID()
+ getMovieID()
+ checkAvailibilty()
```

References: 3.2.2 and 3.2.3

3.4.3 Seat

3.4.1.1 Attributes

- seatID
- availability
- priceSeat

3.4.1.2 Functions

- reserveSeat()
- checkAvailability()
- deleteReservation()
- getSeatID()

```
Seat
- seatID : int
- availability : int
- priceSeat : int

+ reserveSeat()
+ checkAvailibily()
+ deleteReservation()
+ getSeatID()
```

References: 3.2.2 and 3.2.3

3.4.4 Admin

3.4.1.1 Attributes

- adminID

3.4.1.2 Functions

- editMovie()
- editShowtimes()
- getCustomerInfo()

```
Admin
- adminID : int
+ editMovie()
+ editShowtimes()
+ editSeats()
+ getCustomerInfo
```

3.4.5 Ticket

3.4.1.1 Attributes

- ticketID
- priceTicket

3.4.1.2 Functions

- purchaseTicket()
- getTicketID()
- getPrice()

```
Ticket
- ticketID : int
- priceTicket : int
+ purchaseTicket()
+ getTicketID()
+ getPrice()
```

3.4.6 Payment

3.4.1.1 Attributes

- typePayment
- paymentSuccess
- paymentID

3.4.1.2 Functions

- makePayment()
- getPaymentID()
- getTypePayment()

```
Payment
- typePayment : String
- paymentSuccess : bool
- paymentID : int
+ makePayment()
+ getPaymentID()
+ getTypePayment()
```

3.4.7 CustomerInfo

3.4.1.1 Attributes

- customerName
- customerAge
- customerID

3.4.1.2 Functions

- getName()
- getAge()
- getID()

```
CustomerInfo
- customerName : String
- customerAge : int
- customerID : int
+ getName()
+ getAge()
+ getID()
```

3.5 Non-Functional Requirements

3.5.1 Performance

- 95% of user transactions shall be processed in less than a two seconds
- The system shall support 1,000 users at a time

3.5.2 Reliability

- System downtime shall not exceed 2 minutes per day
- There shall not be a system failure more than once each 30 days

3.5.3 Availability

- Movie Ticketing System shall be up 99.99% of the time per year

3.5.4 Security

- Passwords and user IDs will be encrypted
- Payment information will be encrypted

3.5.5 Maintainability

- The Movie Ticketing System will update every 2 months
- It will not take more than 30 seconds

3.5.6 Portability

- The Movie Ticketing System will be able to be used on Windows, Macs, and Linux

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitations, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable to the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2

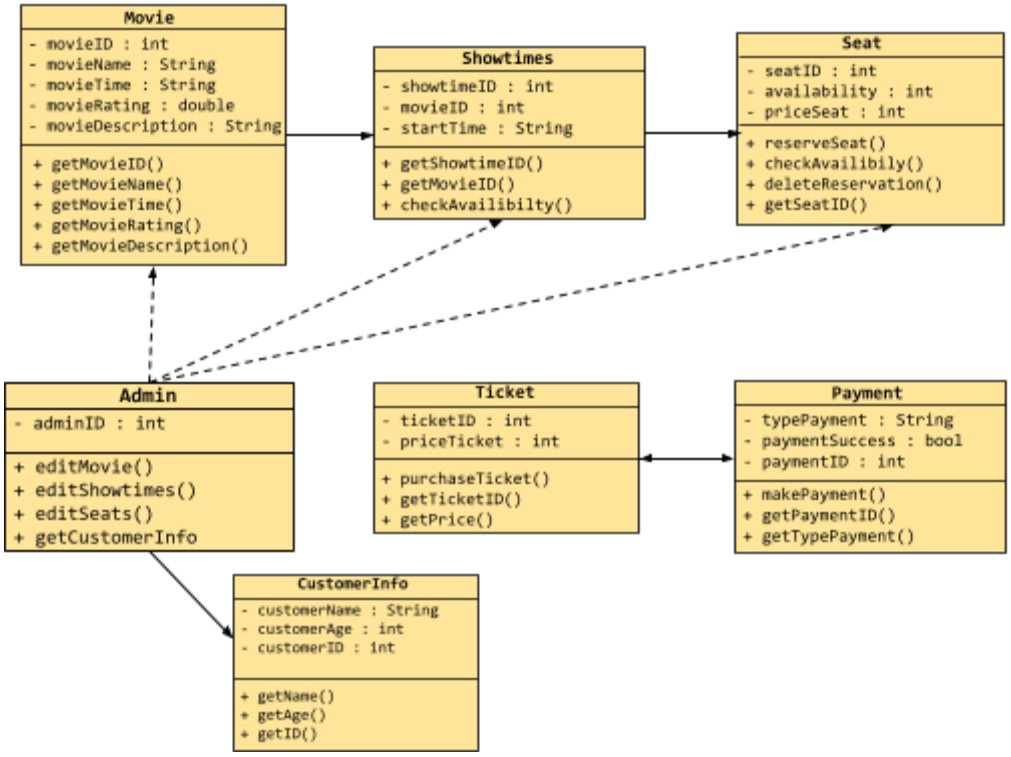
6. Design **Do we add a title page here? Or does the one at the beginning count. Also do we delete the previous sections or fill them in or leave them

6.1 System Description **Make sure this is complete

The Movie Ticketing System is a software application that is attached to a movie theater company's website. It allows the user, either an administrator or a customer, to access movies being played at a certain location, movie showtimes, and seats available (including a map of the seat layout) for a certain movie. It allows customers to pay for a movie ticket. The Movie Ticketing System will authenticate payments and provide a unique movie ticket ID. It allows administrators to edit movies, movie showtimes, and seats available, as well as access customer information. In other words, the administrator can override and edit the Movie Ticketing System.

6.2 Software Architecture Overview **Make sure SWA and UML are right and complete

6.2.1 UML Diagram



6.2.2 UML Diagram Description

Description:

1. Movie Setup
    a. Admin creates / edit movies
    b. Each movie has one or more showtimes
2. Showtime Selection
    a. A customer selects a movie
    b. The system retrieves related showtimes
3. Seat Reservation
    a. For a selected showtime:
        i. Available seats are displayed
        ii. Customer selects seats
        iii. reserveSeats() updates
4. Ticket Creation
    a. After seat reservation:
        i. A ticket is created
        ii. Ticket stores price
5. Payment
    a. Ticket is linked to Payment
        i. makePayment() processes transaction
        ii. if paymentSuccess() true:
        iii. Ticket purchase is confirmed
6. Admin
    a. Edit movies

b. Modify showtimes
c. Adjust seats
d. View customer information

6.3 Development Plan and Timeline

6.3.1 List of Tasks **Is this ok? Or is it more supposed to be making the Movie Ticketing System

1. Implement a fully tested movie class with methods
2. Implement a fully tested showtime class with methods
3. Implement a fully tested seats class with methods
4. Implement a fully tested admin privileges class
5. Implement movie database
6. Implement customer information database
7. Implement a fully tested customer information class with methods
8. Implement a fully tested payment class with methods
9. Implement a fully tested ticket class with methods
10. Connect movie theater website to Movie Ticketing System
11. Implement connection with payment system
12. Do a final run through to make sure there are no bugs

6.3.2 Timeline

- March- Finish movie, showtime, seats, and admin class and have them fully tested
- April- Implement the movie and customer information databases, and finish the customer information, payment, and ticket classes (fully tested)
- May- Connect the movie theater website to Movie Ticketing System, connect to a payment system, and run through to make sure there are no bugs

6.3.3 Task Partitioning and Team Member Responsibilities

- Ella:
  - March- Implement movie class and have it fully tested
  - April- Implement movie database and customer information class (fully tested)
  - May- Connect the movie theater website to the Movie Ticketing System
- Ruhi:
  - March- Implement showtime class and have it fully tested
  - April- Implement customer information database and payment class (fully tested)
  - Connect to a payment system
- Elisa:
  - March- Implement seats and admin classes and have them fully tested
  - April- Implement ticket class and have it fully tested
  - Do a final run through to make sure there are no bugs